LAPORAN PRAKTIKUM

Implementasi Komunikasi MQTT Menggunakan Python dan Mosquitto Broker

Studi Kasus: Smart Room Monitoring

Disusun oleh:

ACHMAD FIKY AKBAR

235150301111043

Dosen Pengampu:

Sabriansyah Rizqika Akbar, S.T., M.Eng., Ph.D.

Mata Kuliah: Cyber Physical System

Program Studi Teknik Informatika

Fakultas Ilmu Komputer

Universitas Brawijaya

2026

BAB I — Pendahuluan

1.1 Latar Belakang

Cyber Physical System (CPS) merupakan sistem yang mengintegrasikan komputasi, jaringan, dan proses fisik, di mana perangkat fisik atau sensor saling bertukar data melalui jaringan untuk dipantau dan dikendalikan. Salah satu tantangan utama pada CPS adalah pertukaran data yang efisien antara banyak perangkat dengan keterbatasan bandwidth dan daya.

MQTT (Message Queuing Telemetry Transport) adalah protokol komunikasi ringan berbasis pola publish–subscribe yang dirancang khusus untuk kondisi tersebut. Pada MQTT, pengirim pesan (publisher) dan penerima (subscriber) tidak berkomunikasi secara langsung, melainkan melalui perantara bernama broker. Pemisahan (decoupling) ini membuat sistem lebih fleksibel dan mudah diperluas.

Laporan ini mendokumentasikan implementasi komunikasi MQTT menggunakan bahasa Python dengan pustaka paho - mqtt dan Eclipse Mosquitto sebagai broker, dengan studi kasus Smart Room Monitoring. Sistem ini mensimulasikan sensor suhu, kelembapan, dan gerak pada beberapa ruangan yang mengirim data ke aplikasi pemantauan melalui broker.

1.2 Tujuan

1. Menjelaskan konsep komunikasi MQTT dalam Cyber Physical System.
2. Menginstal dan menjalankan Mosquitto Broker sebagai message broker.
3. Membuat program publisher dan subscriber menggunakan Python.
4. Menguji pengiriman pesan dengan variasi QoS 0, 1, dan 2.
5. Mendesain struktur topik MQTT yang merepresentasikan sistem CPS.
6. Menggunakan wildcard topic '+' dan '#' pada subscriber.
7. Menganalisis perbedaan perilaku pengiriman pesan berdasarkan QoS dan struktur topik.

BAB II — Desain Sistem

2.1 Deskripsi Studi Kasus

Smart Room Monitoring adalah sistem pemantauan kondisi ruangan secara real-time. Setiap ruangan dilengkapi tiga jenis sensor yang datanya dikirim secara berkala ke broker, lalu diteruskan ke aplikasi pemantauan (dashboard).

Sensor	Keterangan
Temperature	Suhu ruangan dalam satuan derajat Celcius
Humidity	Kelembapan udara dalam persen (%)
Motion (PIR)	Deteksi gerak: DETECTED / NONE

2.2 Ketentuan Teknis

Komponen	Spesifikasi
Broker	Eclipse Mosquitto 2.x
Bahasa	Python 3.x
Library	Paho - mqtt 2.x
Protokol	MQTT (port 1883)
Host	localhost

2.3 Struktur Topik

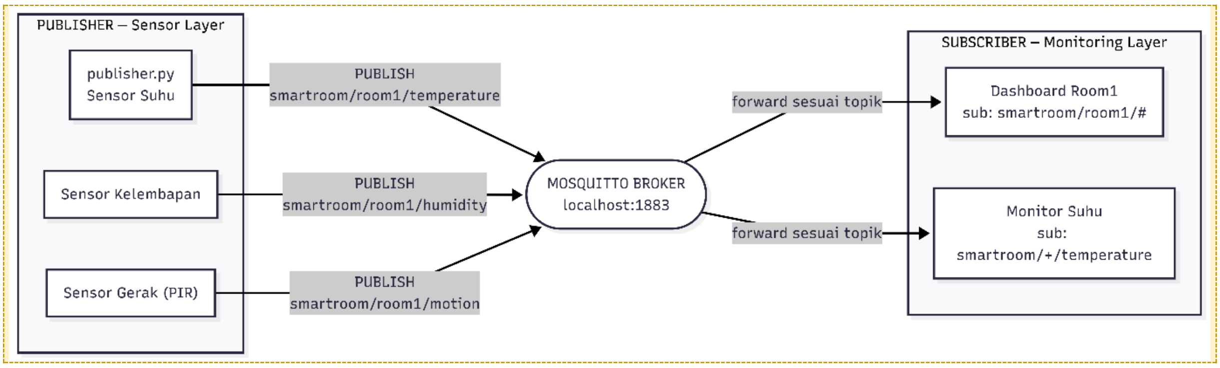
Topik dirancang secara hierarkis tiga tingkat agar mendukung penggunaan wildcard:

```
smartroom/<ruangan>/<sensor>
smartroom/room1/temperature
smartroom/room1/humidity
smartroom/room1/motion
smartroom/room2/temperature
```

Struktur ini memungkinkan, misalnya, subscriber menangkap suhu seluruh ruangan dengan 'smartroom/+/temperature', atau seluruh sensor satu ruangan dengan 'smartroom/room1'.

BAB III — Gambar Model Komunikasi MQTT

Gambar berikut menggambarkan pola publish –subscribe pada sistem. Publisher (sensor) mengirim pesan ke broker pada topik tertentu, dan broker meneruskannya hanya kepada subscriber yang berlangganan topik yang cocok.



Gambar 3.1 Model komunikasi publish–subscribe sistem Smart Room Monitoring.

BAB IV — Implementasi Program

Implementasi terdiri atas lima skenario pengujian. Broker dijalankan dalam mode verbose agar seluruh aktivitas koneksi, subscribe, publish, dan handshake QoS dapat diamati pada log.

```
mosquitto -v -c mosquitto.conf
```

Isi file konfigurasi mosquitto.conf:

```
listener 1883
allow_anonymous true
```

4.1 Skenario 1 — Komunikasi Dasar Publisher–Subscriber

Publisher mengirim data suhu setiap 2 detik ke topik smartroom/room1/temperature, dan subscriber yang berlangganan topik tersebut menerima setiap pesan.

publisher.py

```
import paho.mqtt.client as mqtt
import time, random

BROKER, PORT = "localhost", 1883
TOPIC = "smartroom/room1/temperature"

def on_connect(client, userdata, flags, reason_code, properties):
    print("Terhubung" if reason_code == 0 else f"Gagal: {reason_code}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.connect(BROKER, PORT, 60)
client.loop_start()

while True:
    suhu = round(random.uniform(24.0, 32.0), 2)
    client.publish(TOPIC, suhu)
    print(f"[DIKIRIM] {TOPIC} -> {suhu}")
    time.sleep(2)
```

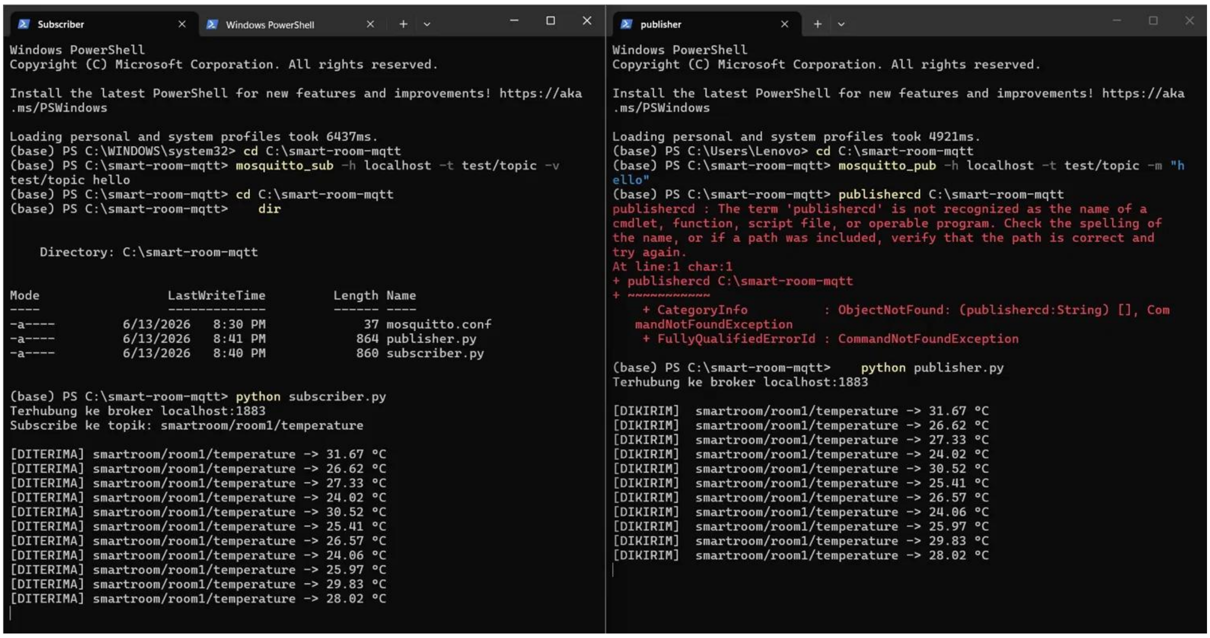
subscriber.py

```
import paho.mqtt.client as mqtt
BROKER, PORT = "localhost", 1883
TOPIC = "smartroom/room1/temperature"

def on_connect(client, userdata, flags, reason_code, properties):
    client.subscribe(TOPIC)

def on_message(client, userdata, msg):
    print(f"[DITERIMA] {msg.topic} -> {msg.payload.decode()}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message
client.connect(BROKER, PORT, 60)
client.loop_forever()
```



Analisis: nilai yang diterima subscriber identik dengan yang dikirim publisher, membuktikan pola publish–subscribe dasar berjalan melalui broker.

4.2 Skenario 2 — Pengiriman Data dengan QoS Berbeda (0, 1, 2)

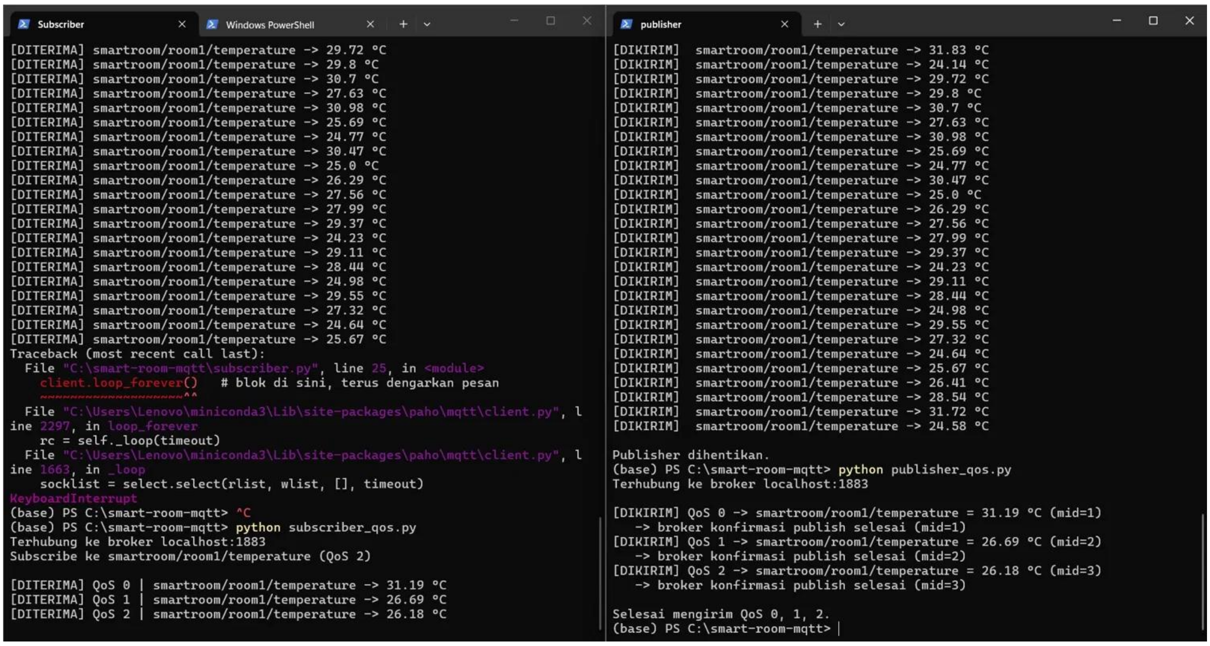
Publisher mengirim tiga pesan berturut-turut menggunakan QoS 0, 1, dan 2. Perbedaan jaminan pengiriman paling jelas terlihat pada log broker.

QoS	Jaminan	Mekanisme handshake
0	at most once	PUBLISH (tanpa balasan)
1	at least once	PUBLISH → PUBACK
2	exactly once	PUBLISH → PUBREC → PUBREL → PUBCOMP

Potongan inti publisher_qos.py:

```
for qos in [0, 1, 2]:
    suhu = round(random.uniform(24.0, 32.0), 2)
    info = client.publish(TOPIC, suhu, qos=qos)
    print(f"[DIKIRIM] QoS {qos} -> {suhu} (mid={info.mid})")
    info.wait_for_publish()
    time.sleep(2)
```

Pada subscriber, topik di-subscribe dengan QoS 2 (client.subscribe(TOPIC, qos=2)) agar dapat menerima pada QoS maksimal.



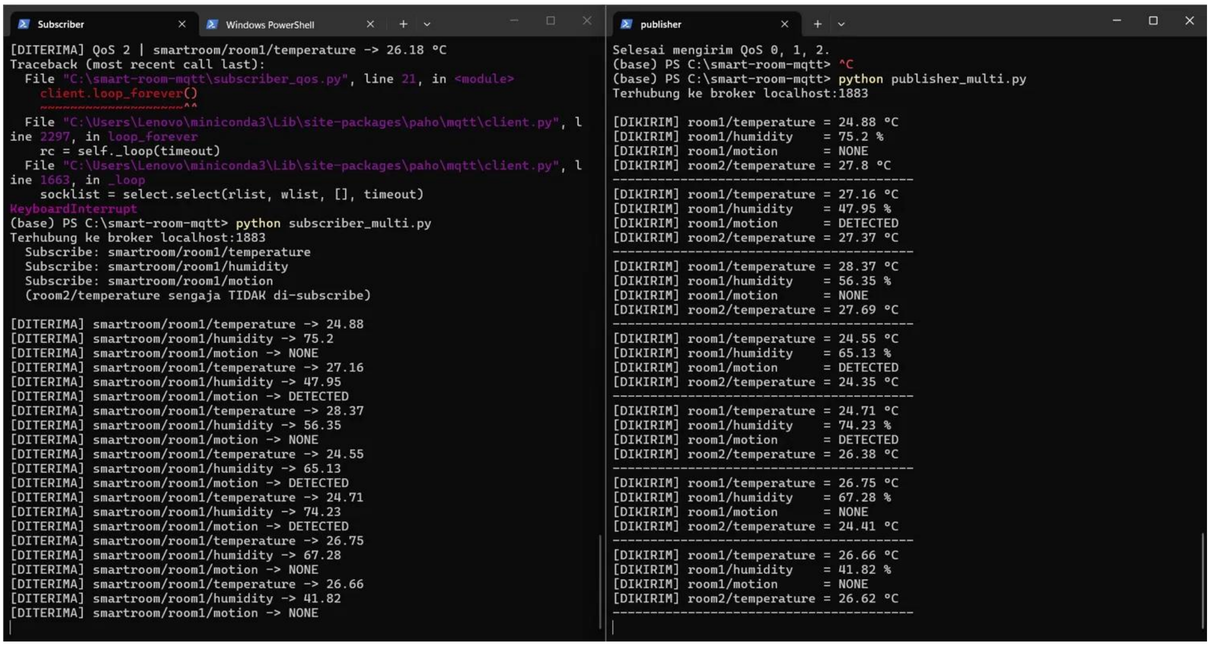
Analisis: QoS 0 hanya menampilkan PUBLISH tanpa balasan; QoS 1 memunculkan PUBACK; QoS 2 menambah rangkaian PUBREC, PUBREL, PUBCOMP. Semakin tinggi QoS, semakin kuat jaminan pengiriman namun semakin banyak pertukaran paket (overhead).

4.3 Skenario 3 — Penggunaan Beberapa Topik

Publisher mengirim ke empat topik (tiga sensor room1 dan satu sensor room2). Subscriber hanya berlangganan tiga topik room1, sehingga smartroom/room2/temperature tidak diterima.

Potongan subscriber_multi.py:

```
TOPICS = [
    ("smartroom/room1/temperature", 0),
    ("smartroom/room1/humidity", 0),
    ("smartroom/room1/motion", 0),
]
client.subscribe(TOPICS)   # subscribe banyak topik sekaligus
```

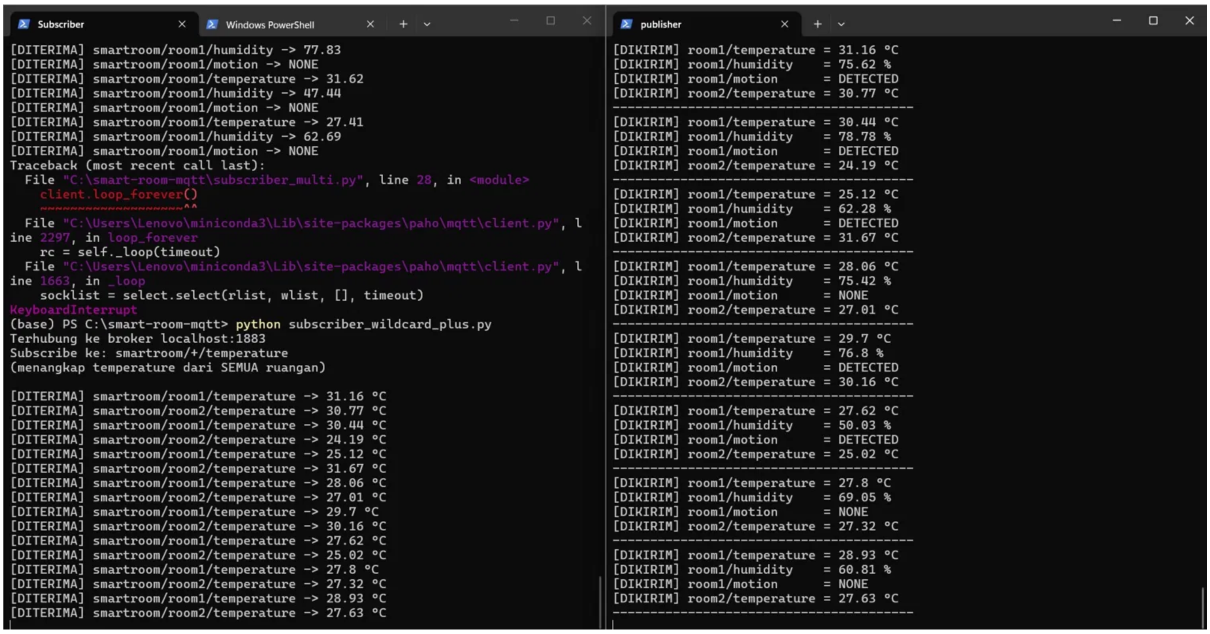


Analisis: meski room2/temperature terus dikirim publisher, pesan tersebut tidak pernah *muncul di subscriber. Hal ini membuktikan broker menyaring dan meneruskan pesan hanya berdasarkan topik yang di-subscribe.*

4.4 Skenario 4 — Penggunaan Wildcard '+'

Wildcard '+' mewakili tepat satu level topik. Subscriber berlangganan smartroom/+/temperature untuk menangkap suhu dari seluruh ruangan tanpa menyebut nama tiap ruangan.

```
TOPIC = "smartroom/+/temperature"
client.subscribe(TOPIC)
```

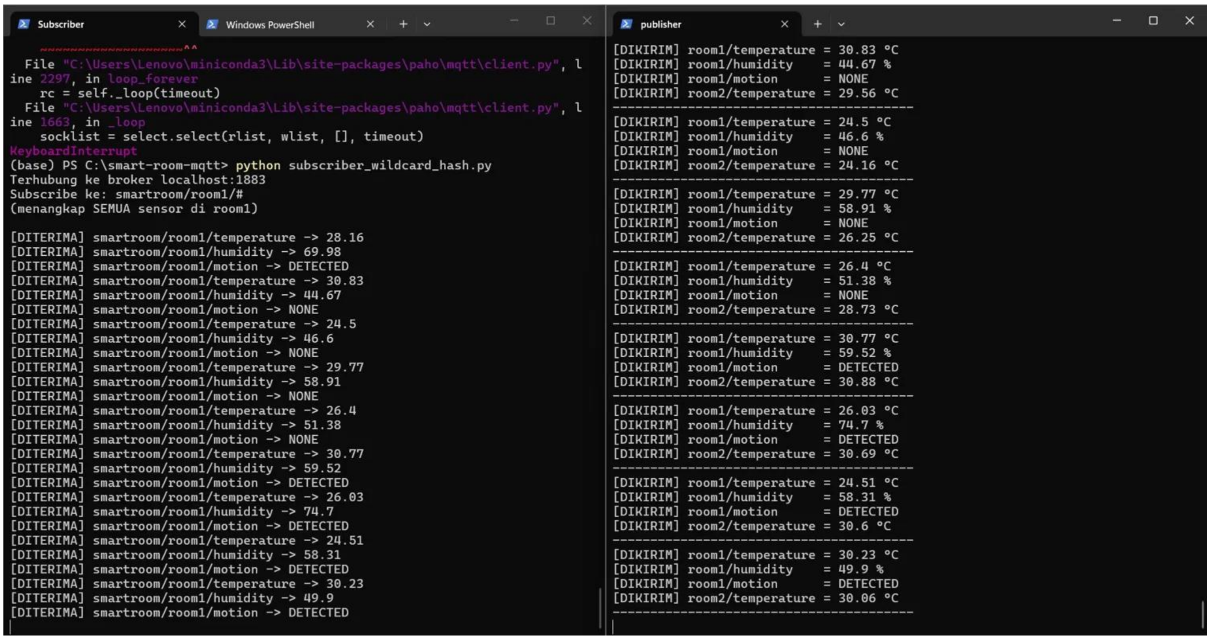


Analisis: subscriber menerima smartroom/room1/temperature dan smartroom/room2/temperature, tetapi tidak menerima humidity maupun motion. Wildcard '+' hanya mencocokkan satu level (posisi ruangan), sedangkan level terakhir tetap harus 'temperature'.

4.5 Skenario 5 — Penggunaan Wildcard '#'

Wildcard '#' mewakili semua level setelahnya dan harus berada di akhir topik. Subscriber berlangganan smartroom/room1/# untuk menangkap seluruh sensor pada room1.

```
TOPIC = "smartroom/room1/#"
client.subscribe(TOPIC)
```



Analisis: subscriber menerima temperature, humidity, dan motion dari room1, namun tidak menerima topik room2. Jika topik diganti menjadi 'smartroom/#', subscriber akan menerima seluruh topik dari semua ruangan.

BAB V — Ringkasan dan Kesimpulan

5.1 Ringkasan Hasil Pengujian

Skenario	Topik Subscriber	Hasil
1	smartroom/room1/temperature	Menerima setiap data yang dipublish
2	smartroom/room1/temperature (QoS 2)	Perbedaan handshake QoS 0/1/2
3	3 topik room1 (eksplisit)	room2/temperature tersaring keluar
4	smartroom/+/temperature	Suhu seluruh ruangan
5	smartroom/room1/#	Semua sensor room1

5.2 Kesimpulan

Implementasi membuktikan bahwa MQTT menyediakan komunikasi publish – subscribe yang terpisah (decoupled) melalui broker, sesuai kebutuhan Cyber Physical System. Tingkat QoS menentukan keseimbangan antara keandalan pengiriman dan overhead jaringan, sedangkan struktur topik berhierarki dengan wildcard '+' dan '#' memberikan fleksibilitas tinggi dalam menyaring dan mendistribusikan pesan tanpa mengubah sisi publisher.